

電腦圖學 Homework #1

A Drawing Panel

黃俊源 01257128 2026-10-05

Contents

1 專案介紹	2	4.3 物件選取與刪除	10	9.4 UTF-8 與 GFNT 資料 . .	18
1.1 作業目標	2	4.4 Stroke Rendering . . .	11	10 已知限制與未來改進	19
1.2 系統概述	2	4.5 Fill Rendering	12	11 心得	19
2 使用者操作說明	3	4.6 座標系統	12	A 自製 GUI Framework	20
2.1 主畫面	3	4.7 Grid	12	A.1 設計動機與範圍	20
2.2 繪圖工具	3	5 文字功能	13	A.2 Application 與 Window	20
2.3 繪圖屬性	3	5.1 FreeGLUT 字型	13	A.3 Element Tree 與 Lay- out System	20
2.4 階層式選單	4	5.2 Text Object 與文字輸入	13	A.4 EventTarget、Bub- bling 與 stopPropa- gation	22
2.5 快捷鍵與滑鼠操作	5	5.3 UTF-8 與中文字元支援 .	14	A.5 Mouse Capture 與 In- put State	22
2.6 視窗與對話框	6	5.4 自訂中文字型支援	14	A.6 Multi-window Lifecycle	22
3 系統架構與設計	6	6 Framebuffer 與輸出	15	B GFNT 自訂點陣字型格式	23
3.1 整體架構	6	6.1 Color Buffer 與畫面還原	15	B.1 設計動機	23
3.2 Application 與 Window	6	6.2 圖片輸出	15	B.2 Binary Layout	23
3.3 Document、Scene 與 Scene Object	7	7 特殊貢獻摘要	15	B.3 Loader 與驗證	24
3.4 Event 與 Input	8	8 成果展示與作業需求驗證	16	B.4 UTF-8 Decoder 與 Glyph Lookup	24
3.5 Command History 與 Undo/Redo	8	8.1 繪圖成果	16	B.5 Bitmap Rendering . . .	25
3.6 Rendering Pipeline . .	9	8.2 進階功能成果	16	C Framebuffer 與輸出實作細節	26
4 繪圖與渲染實作	9	8.3 Requirement Checklist	16	C.1 座標與 Pixel Row	26
4.1 基本圖形	9	9 開發問題與解決方法	18	C.2 擷取與還原狀態	26
4.2 Interactive Drawing 與 Draft	10	9.1 GLUT Callback 與多視 窗狀態	18	C.3 PPM P6 轉換	27
		9.2 Draft 與 Undo / Redo .	18		
		9.3 粗線轉角的幾何例外 . .	18		

1 專案介紹

1.1 作業目標

本次作業的目標是使用 C++17、OpenGL 與 FreeGLUT 實作一套互動式 2D Drawing Panel。使用者可以透過滑鼠與鍵盤繪製基本幾何圖形與文字，並利用階層式選單調整顏色、線寬、點大小、填滿模式與字型等繪圖屬性。此外，程式也需要處理 keyboard、mouse、window 與 motion event，並利用 Color Buffer 保存與恢復畫面內容。

本次完成的繪圖程式命名為 **Graphoria**。除了作業要求的基本繪圖功能外，目前也加入 Select、Undo/Redo、Grid、多視窗、文字輸入對話框與 PPM 圖片輸出等功能。其中 Select 可以選取畫面中最上層的圖形或文字、顯示選取範圍，並透過可復原的 Command 刪除物件。

在實作過程中，我也希望各項功能不要全部集中在 GLUT callback 與全域狀態中，因此將視窗管理、使用者輸入、繪圖資料與畫面渲染分開處理。這些設計除了讓程式較容易整理，也使後續加入多視窗、Undo/Redo 與物件選取等功能時，不必重新修改整個繪圖流程。

1.2 系統概述

程式啟動後會先載入繪圖所需的字型資源，再建立主要繪圖視窗。主畫面中央為 Canvas，底部狀態列顯示目前使用的工具、滑鼠位置與畫布大小；在 Canvas 上按下滑鼠右鍵則可以開啟階層式選單，切換繪圖工具或調整各種繪圖屬性。

使用者進行繪圖時，工具會先建立暫時的 draft，並隨著滑鼠移動更新內容，讓使用者可以即時看到目前的繪製結果。當操作完成後，物件才會正式加入 Scene，並透過 Command 記錄這次修改，因此 Undo/Redo 不需要知道各種工具實際如何收集滑鼠輸入。

畫面顯示時，程式會依序繪製 Grid、Scene 中已完成的物件、尚未完成的工具預覽，以及目前的選取範圍。滑鼠與鍵盤輸入則先由視窗轉換成程式內部的 Event，再交由對應的 UI 元件或 Canvas 處理。整體架構與各模組之間的關係將在後續章節進一步說明。

2 使用者操作說明

2.1 主畫面

程式啟動後會建立主要繪圖視窗，中央區域為 **Canvas**，使用者可以直接在其中進行繪圖與物件選取。視窗底部的狀態列會顯示目前使用的工具、繪圖樣式、滑鼠在 **Canvas** 中的位置，以及目前 **Canvas** 的尺寸。預設工具為 **Pencil**，**Grid** 則預設使用 **Lines** 模式。

在 **Canvas** 上按下滑鼠右鍵可以開啟階層式選單，切換工具或調整各種繪圖屬性。圖 1 為程式的主畫面。

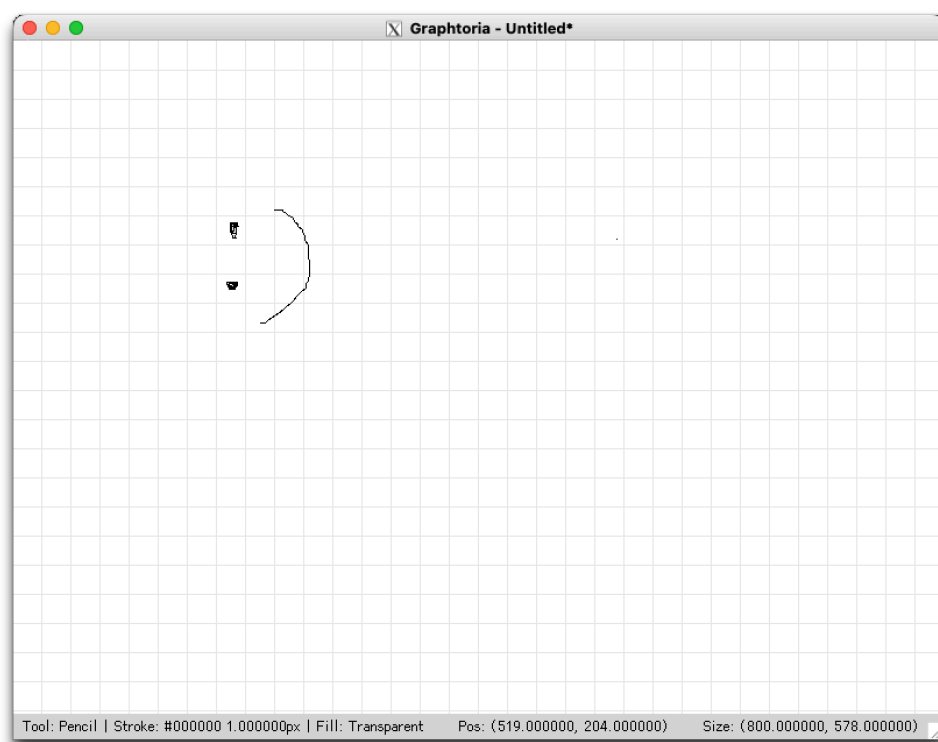


Figure 1: 繪圖程式主畫面

2.2 繪圖工具

目前提供 **Select**、**Point**、**Pencil**、**Line**、**Rectangle**、**Ellipse**、**Polygon** 與 **Text** 等工具，各工具的主要操作方式整理於表 1。

2.3 繪圖屬性

程式可以分別設定圖形的 **Stroke**、**Fill**、**Point** 與 **Text** 屬性。

Table 1: 繪圖工具與操作方式

工具	操作方式
Select	單擊物件進行選取；Delete 或 Backspace 可刪除目前選取的物件，Esc 或點擊空白處可取消選取。
Point	在 Canvas 上單擊建立一個點，點的大小由目前的 Point Size 決定。
Pencil	按住滑鼠左鍵拖曳即可自由繪製，放開滑鼠後完成目前的 Path。
Line	按住滑鼠左鍵拖曳決定線段的起點與終點；繪製時按住 Shift 可限制為水平、垂直或 45° 方向。
Rectangle	按住滑鼠左鍵拖曳決定矩形的兩個對角；按住 Shift 可限制為正方形。
Ellipse	按住滑鼠左鍵拖曳決定橢圓的外接矩形；按住 Shift 可限制為正圓。
Polygon	單擊滑鼠左鍵依序加入頂點，雙擊或按下 Enter 完成圖形。Backspace 可移除上一個頂點，Esc 則取消本次繪製。
Text	在欲放置文字的位置單擊後開啟文字輸入對話框，輸入完成後將文字加入 Canvas。

Stroke 可調整顏色、線寬，以及 MITER、BEVEL、ROUND 三種 Line Join；線段端點則提供 BUTT、SQUARE、ROUND 三種 Line Cap。Fill 支援 OUTLINE、FILLED 與 ADVANCED 三種模式，其中 ADVANCED 會分別處理填色與自製的粗線幾何。

Point Size 與 Stroke Width 可以透過右鍵選單調整，也可以利用中括號快捷鍵快速增加或減少大小。文字則可以選擇 FreeGLUT 或自訂的點陣字型。

2.4 階層式選單

在 Canvas 上按下滑鼠右鍵即可開啟主選單。選單依功能分成數個子選單：

- **File**：建立文件、開啟文件、儲存及輸出圖片。
- **Edit**：Undo、Redo 與清空目前畫布。
- **Tools**：切換目前使用的繪圖或選取工具。
- **Stroke**：設定線條顏色、寬度、Join 與 Cap。
- **Fill**：設定填滿模式與填色。
- **Point**：調整 Point Size。
- **Text**：選擇文字字型。
- **Grid**：切換格線的顯示模式。

常用功能也提供鍵盤快捷鍵，因此不需要每次都透過選單操作。

2.5 快捷鍵與滑鼠操作

程式內部以 Primary 表示各作業系統慣用的主要修飾鍵¹。例如表 2 中的 Primary+S，在 macOS 為 Command+S，在其他平台則為 Ctrl+S。

Table 2: 程式快捷鍵

快捷鍵	功能
Primary+Z	Undo；若目前仍有未完成的 draft，則先取消此次操作。
Primary+Shift+Z	Redo。
C	清空目前畫布。
G	依序切換 Lines、Dots 與 None 三種 Grid 模式。
Primary+N	建立新文件；若目前內容尚未儲存，會先顯示確認視窗。
Primary+Shift+N	建立新的繪圖視窗。
Primary+O	開啟文件。
Primary+S	儲存目前文件。
Primary+Shift+S	另存新檔。
Primary+E	將目前 Canvas 輸出為 PPM P6 圖片。
Primary+R	重新整理目前視窗，並重設鍵盤與滑鼠輸入狀態。
0	選擇 Select。
1	選擇 Pencil。
2	選擇 Line。
3	選擇 Rectangle。
4	選擇 Ellipse。
5	選擇 Polygon。
6	選擇 Text。
[	Stroke Width 與 Point Size 減少 1 px，最小為 1 px。
]	Stroke Width 與 Point Size 增加 1 px。
Shift+[	Stroke Width 與 Point Size 減少 5 px，最小為 1 px。
Shift+]	Stroke Width 與 Point Size 增加 5 px。

滑鼠左鍵主要用於繪圖與物件選取，右鍵則用來開啟階層式選單。使用 Select 時，若多個物件互相重疊，程式會優先選取視覺上最上層的物件；點擊空白處或按下 Esc 即可取消目前的選取。

Line、Rectangle 與 Ellipse 在拖曳期間可以按住 Shift 限制形狀；Polygon 則可以利用雙擊或 Enter 完成，並可使用 Backspace 移除上一個頂點、Esc 取消本次繪製。

¹macOS 對應 Command，Windows 與 Linux 對應 Ctrl。

2.6 視窗與對話框

程式可以同時建立多個繪圖視窗，每個視窗都有獨立的 Document 與 Undo/Redo 紀錄，彼此之間的繪圖內容不會互相影響。

文字輸入、檔名輸入與確認操作會另外開啟對話框。當 Modal Dialog 開啟時，其他繪圖視窗會暫停接收使用者輸入，避免在對話框尚未處理完成時誤操作背景視窗。

若目前文件仍有尚未儲存的修改，執行 New、Load 或 Close 等可能捨棄目前內容的操作時，程式會先顯示確認視窗，再由使用者決定是否繼續。

3 系統架構與設計

3.1 整體架構

為了避免繪圖、事件處理與 OpenGL 呼叫全部集中在 GLUT callback 中，程式將主要功能拆分為視窗管理、文件資料、繪圖工具與畫面渲染等部分，整體架構如圖 2 所示。

Application 負責建立與管理各個視窗，PaintWindow 則組合繪圖介面、選單、快捷鍵與目前的 Document。使用者在 Canvas 上完成一次操作後，產生的繪圖物件會交由 Document 修改 Scene；顯示時再由 Renderer 讀取 Scene 中的內容並繪製到畫面上。

這樣的分工讓使用者操作、文件資料與渲染流程彼此分開。例如 Undo/Redo 只需要處理文件內容的修改，Renderer 也不需要知道物件是透過哪一種工具建立的。

3.2 Application 與 Window

Application 負責 FreeGLUT 的初始化，以及所有視窗的建立與生命週期管理。除了主要的 PaintWindow 外，文字輸入、確認視窗與圖片輸出所使用的視窗也共用相同的管理方式。各視窗類別的繼承關係與 Application 的管理關係如圖 3 所示。

FreeGLUT 使用 C-style callback，因此無法直接將 callback 綁定到某一個 C++



Figure 2: 繪圖程式的整體架構

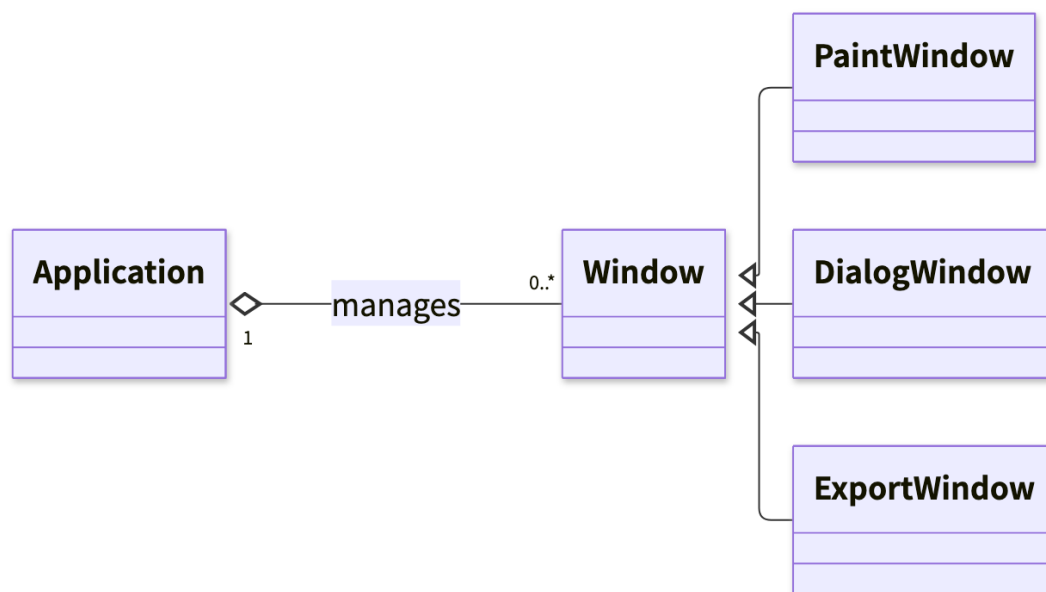


Figure 3: Application 與視窗類別關係

物件²。為了解決這個問題，每個 Window 建立時都會記錄自己的 GLUT window ID；callback 發生後，再利用目前的 window ID 找回對應的 Window instance，最後將輸入交給該視窗處理。

Menu 的處理方式也類似。FreeGLUT 的 menu callback 只提供整數 ID，程式再透過這個 ID 找回對應的選單項目並執行其動作。

較完整的 callback routing、Modal Dialog 與多視窗生命週期整理於附錄 A。

3.3 Document、Scene 與 Scene Object

Document 代表一份繪圖文件，其中保存檔案資訊、Canvas 尺寸、Scene 與 CommandHistory。新增、刪除或清除繪圖內容時，都會透過 Document 修改，而不是由繪圖工具直接操作 Scene。

Scene 則依照繪製順序保存所有繪圖物件。目前主要分為 ShapeObject 與 TextObject：前者用來表示 Point、Line、Rectangle、Ellipse、Path 與 Polygon，後者則保存文字位置、UTF-8 字串與文字樣式。

繪圖工具在操作完成後只需要產生對應的 Scene Object，再交由 Document 建立 Command 並加入 Scene。因此 Scene 本身只負責保存資料，不需要知道 UI、工具或 OpenGL 的存在。

拖曳期間顯示的 draft 與 Select 的選取框則屬於暫時的互動狀態，不會加入 Scene。

²Non-static member function 具有隱含的 **this** 參數，其函式型別與 FreeGLUT 所要求的 callback signature 不相同。

只有當繪圖操作真正完成，或使用者確定刪除物件時，才會產生對文件內容的修改，因此也不會在操作過程中產生多餘的 Undo 紀錄。

圖 4 整理了 PaintWindow、Document 與場景物件之間的擁有與繼承關係。

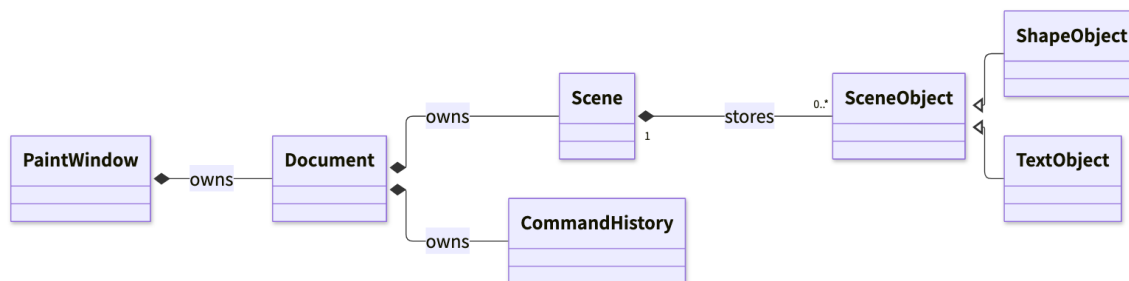


Figure 4: Document、Scene 與 Scene Object 類別關係

3.4 Event 與 Input

FreeGLUT 提供的 keyboard、mouse 與 motion callback 會先由 Window 轉換成程式內部使用的 Event。Event 中除了事件本身的資料外，也包含目前的 keyboard 與 mouse state，讓後續元件可以用一致的方式判斷滑鼠位置、按鍵與 Shift 等狀態。

滑鼠事件會先經過 hit test 找出對應的 Element。若目前已有 Element 取得 mouse capture，則事件會優先交給該 Element，使拖曳操作即使暫時離開原本區域仍能正常收到 MouseUp。

因此工具本身不需要直接處理 GLUT 的原始 callback，只需要回應整理過後的 Event。Event bubbling、focus、mouse capture 與 shortcut matching 等細節則收錄於附錄 A。

3.5 Command History 與 Undo/Redo

為了支援 Undo/Redo，所有會修改文件內容的操作都會封裝成可復原的 Command。目前新增、插入、移除物件與清空畫布等操作，都會經由 CommandHistory 執行。

執行新的 Command 後，該操作會加入 undo stack；使用 Undo 時會復原這次修改，並將 Command 移到 redo stack；Redo 則再次執行同一筆 Command。若 Undo 後又執行新的操作，原本的 redo stack 便會被清除。

每筆紀錄另外保存操作前後的状态 ID，用來判斷目前文件是否仍與最近一次儲存的狀態相同。移除物件時也會記錄它原本在 Scene 中的位置，因此 Undo 後可以恢復原來的前後層級。

圖 5 整理了 Command History 的主要流程。

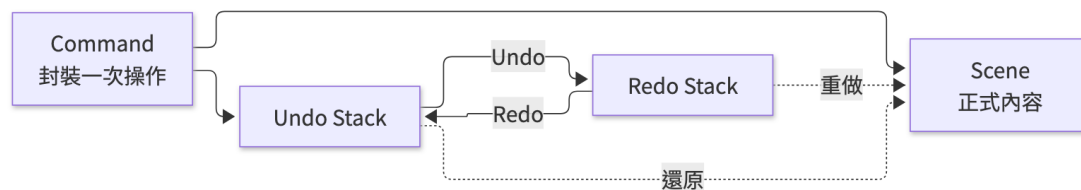


Figure 5: Command、Undo 與 Redo 的資料流

3.6 Rendering Pipeline

當視窗需要重新顯示時，Element tree 會依照各自的局部座標進行 render。輪到 Canvas 時，CanvasRenderer 會先繪製 Grid，接著按照 Scene 中的順序繪製正式物件，再顯示目前工具尚未完成的 draft，最後才繪製 Select 的選取框。

對於 Scene 中的物件，SceneRenderer 會先判斷物件種類，再交給對應的 Shape 或 Text renderer。圖形部分負責 Stroke 與 Fill，文字部分則處理字型與 Glyph 的顯示，最後才轉換成實際的 OpenGL 繪圖命令。

這樣的繪製順序可以讓 Grid 保持在背景，正式內容位於中間，而工具預覽與選取提示始終顯示在最上層。

整體 Rendering Pipeline 如圖 6 所示。

4 繪圖與渲染實作

4.1 基本圖形

每種幾何圖形都能提供 Renderer 所需的頂點。Line 回傳兩端點，Rectangle 回傳四個角，Ellipse 以固定 64 個 segment 取樣，Path 與 Polygon 則使用工具收集的點。Point 是較簡單的特例，直接以 GL_POINTS 和目前的 point size 繪製。Renderer 因此只需要處理完成後的 geometry，不用知道前面是單擊、拖曳還是多次點擊。

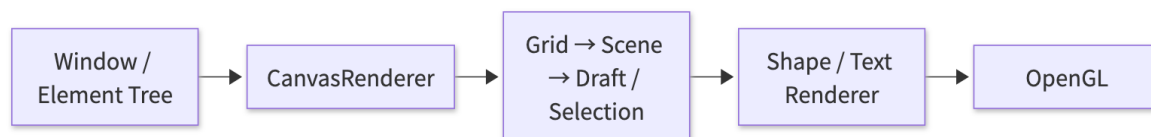


Figure 6: 畫面渲染流程

4.2 Interactive Drawing 與 Draft

繪圖工具不會在 Mouse Down 時立刻修改 Document，而是先建立一個 draft。接下來的 Motion 只改變 draft 的座標並要求重畫，所以使用者可以立即看到預覽。直到 Mouse Up、DoubleClick 或 Enter 表示操作完成，Canvas 才把新物件交給 Document 與 Command History。圖 7 是 DragTool 和 PolygonTool 共用的主要流程。

切換工具時，目前工具會先處理尚未完成的 draft。若使用者在互動過程中按下 Undo，程式先取消 draft，而不是立刻復原 Scene 中上一個物件。這讓「取消這次尚未完成的動作」和「復原前一次已完成的動作」不會混在一起。

4.3 物件選取與刪除

SelectTool 由 Scene 尾端向前搜尋，也就是先檢查最後繪製的物件。一旦命中就停止搜尋，因此圖形重疊時會優先選到視覺上的最上層物件。文字物件以字型計算出的寬高建立矩形，點擊座標落在矩形內即為命中。

圖形物件先轉成頂點序列。對於「封閉且有可見填色」的圖形，程式以 ray-crossing (even-odd) 測試點擊位置是否在多邊形內；每當向右的水平射線穿過一條邊就切換一次內外狀態。這使 Rectangle、Polygon，以及由 64 個 segment 近似的 Ellipse，都能從填色區域內部選取。

只有外框或開放路徑時，則計算點擊點 P 到每條線段 $\overline{AB}$ 的最短距離。令 $\vec{v} = B - A$ ，投影參數與距離為

$$t = \text{clamp}\left(\frac{(P - A) \cdot \vec{v}}{\vec{v} \cdot \vec{v}}, 0, 1\right), \quad d(P, \overline{AB}) = \|P - (A + t\vec{v})\|.$$

若 $d \leq \tau$ 即判定命中，其中容差

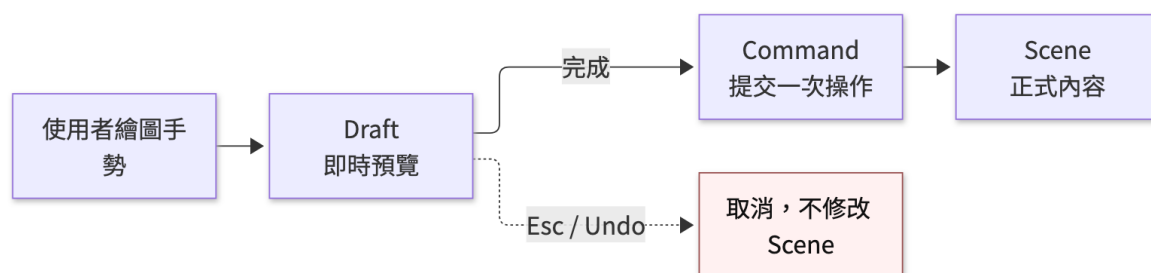


Figure 7: 繪圖工具與草稿的生命週期

$$\tau = \max\left(\frac{w}{2} + 3, 5\right) \text{ px}$$

會隨畫筆寬度 w 增加，但至少保留 5 px，讓細線也容易點選；Point 物件則以 point size 代替 w 。封閉圖形會額外檢查最後一點到第一點的邊，長度為零的線段則退化為與端點的距離比較。

選取後，overlay 會在 Scene 與 draft 之上畫出 axis-aligned bounding box。該範圍由頂點的最小與最大座標取得，再向外擴張 $\max(w/2, 2)$ px，避免粗畫筆被選取框截斷。

Delete 或 Backspace 會產生 RemoveObjectAction，再由 Document 建立記住原始索引的 RemoveObjectCommand。因此刪除可進入歷史紀錄，Undo 也能恢復原本的繪製層級。

4.4 Stroke Rendering

Advanced mode 不直接依賴 OpenGL 的內建粗線，而是自行產生 triangle geometry。假設相鄰三點為 A 、 B 、 C ，先計算兩段的單位方向向量

$$\vec{u} = \frac{B - A}{\|B - A\|}, \quad \vec{v} = \frac{C - B}{\|C - B\|}.$$

接著將方向向量旋轉 90° 得到法向量 $\vec{n}_0$ 、 $\vec{n}_1$ ，並以半寬 $h = w/2$ 建立左右兩側的 offset line，例如第一段的邊界會通過 $A \pm h\vec{n}_0$ 與 $B \pm h\vec{n}_0$ 。程式求出兩組 offset line 的交點，再用二維 cross product 判斷轉向：

$$\vec{u} \times \vec{v} = u_x v_y - u_y v_x.$$

Canvas 的 Y 軸向下，因此程式中的結果大於 0 表示向右轉，小於 0 表示向左轉；這個符號接著用來選擇哪一側是外側。MITER 使用外側交點，BEVEL 用一個 triangle 截去尖角，ROUND 則補上一段 triangle fan arc。圖 8a 比較三種結果。

開放路徑的端點只有一個方向。BUTT 直接停在原端點；SQUARE 沿線段方向再延伸 h ；ROUND 則以端點為圓心補上半圓。三種結果如圖 8b。這些端點不是交給 `glLineWidth()` 決定，而是由同一套 geometry 計算產生。

當兩段線形成很尖的角度時，外側 offset line 的交點可能離 B 很遠。程式將這段距離記為 miter length m ，並與 hL 比較，其中 L 是 miter limit，目前預設為 4：

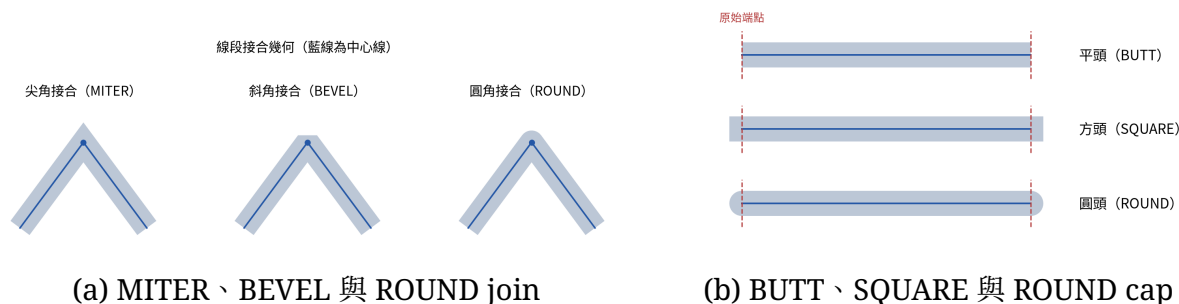


Figure 8: 自行產生的筆畫接合與端點；細線或虛線表示中心線

使用 MITER 的條件： $m \leq hL$.

若 $m > hL$ ，即使使用者選擇 MITER，程式仍改用 BEVEL，避免出現過長的尖刺。圖 9 顯示這項判斷。

程式也處理平行、近 180 度折返與過遠 inner intersection。若 offset line 無法穩定求交，Join 退回 BEVEL 或 ROUND；inner intersection 超過附近 segment 長度與半寬推導出的限制時，改用原 vertex，避免短筆畫出現異常 triangle。

4.5 Fill Rendering

OUTLINE 與 FILLED 模式直接使用 OpenGL primitive 和 `glPolygonMode()`。ADVANCED 模式則把填色與描邊分開：封閉圖形先填入 Fill color，再畫出自行計算的 Stroke geometry。若 Fill color 完全透明或頂點少於三個，就跳過填色。目前填色仍使用 `GL_POLYGON`，因此凹多邊形可能無法得到正確結果，這部分尚未加入 triangulation。

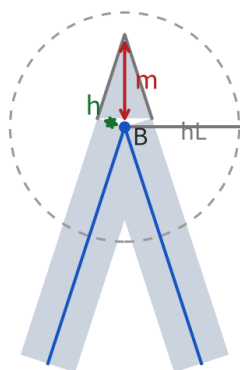
4.6 座標系統

程式以 `glOrtho(0, width, height, 0, -1, 1)` 將 UI、滑鼠與 Scene 統一為左上角原點。Canvas 再沿 parent chain 將視窗座標轉成局部座標，因此工具收到的位置、Scene 儲存的頂點與狀態列顯示一致；只有 framebuffer pixel I/O 仍採 OpenGL 的左下角順序，細節見附錄 C。

4.7 Grid

Grid 只提供位置參考，不會加入 Scene 或改變物件頂點。Lines 與 Dots 模式都以 25 px 為間距，None 則不繪製；CanvasRenderer 會先畫 Grid，再畫 Scene、draft 與選取框，使格線保持在背景。

頂點夾角 = 36°
 $m/h = 3.2 \leq L = 4$ ：使用尖角接合 (MITER)



頂點夾角 = 18°
 $m/h = 6.4 > L = 4$ ：改用斜角接合 (BEVEL)

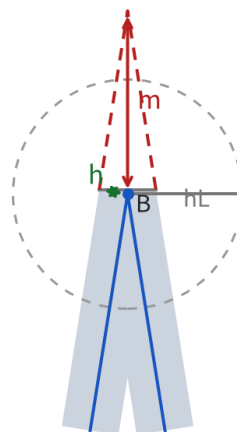


Figure 9: Miter limit 的實際轉角案例；灰底表示筆畫寬度，藍線為中心線

5 文字功能

5.1 FreeGLUT 字型

文字物件可以使用 FreeGLUT 的 Bitmap 與 Stroke 字型；Renderer 會處理 baseline，以及 Stroke 字型和 Canvas 相反的 Y 軸方向。這些內建字型主要用來顯示 ASCII。

5.2 Text Object 與文字輸入

選擇 Text tool 並點擊 Canvas 後，程式會開啟 Modal Input Dialog。只有在使用者確認文字後，才會建立 TextObject 並加入 Scene；如果取消輸入，Scene 不會留下空的文字物件。Dialog 只負責收集字串，實際修改 Document 的工作仍由 PaintWindow 完成。

輸入框內部以 Unicode code point 為編輯單位，對外仍使用 UTF-8 字串，因此 Backspace、Delete 與方向鍵不會切斷中文字的 bytes。FreeGLUT 的一般 keyboard callback 只能直接提供單一 byte，所以程式另提供反引號加四位十六進位數字的測試輸入，例如 ``4f60` 會輸入「你」。Caret 與 UTF-8 bytes 的完整對應見附錄 B。

圖 10 是從程式實際輸出的圖片裁切而成，可確認多個中文字都能找到對應 Glyph，字元順序與基線也符合輸入內容。

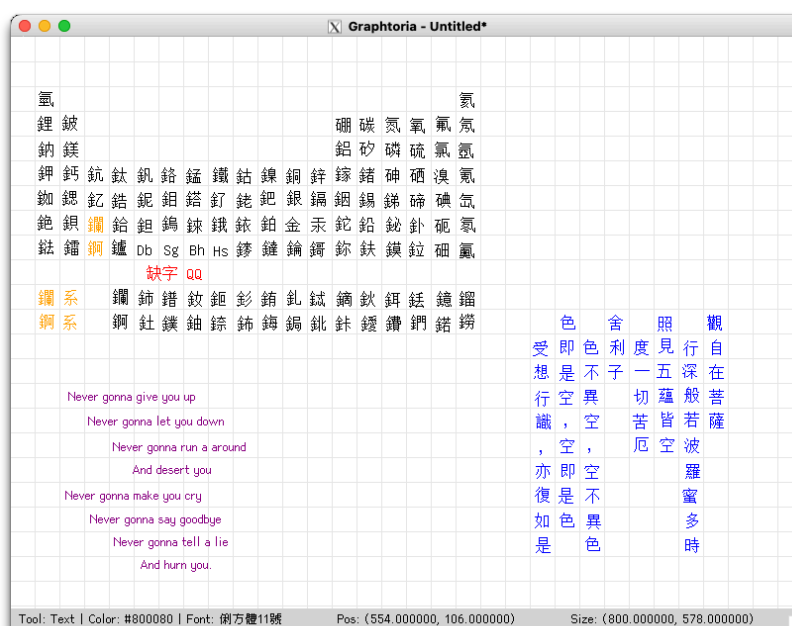


Figure 10: GFNT 在程式中測試多個中文字形的實際輸出結果

5.3 UTF-8 與中文字元支援

UTF-8 decoder 支援 1–4 byte sequence，遇到不合法輸入時以 U+FFFD 取代。輸入框先將 UTF-8 轉成 code point 編輯，顯示時再重新編碼；TextRenderer 也使用相同流程逐字取得 glyph。

5.4 自訂中文字型支援

為了顯示 FreeGLUT 沒有涵蓋的中文字，我設計了 GFNT bitmap font。Renderer 從 UTF-8 取得 code point，查找 glyph 的尺寸、bearing、advance 與 bitmap，再組成 RGBA pixels 交給 OpenGL。執行期流程如圖 11；binary layout、驗證與 fallback 規則收錄於附錄 B。

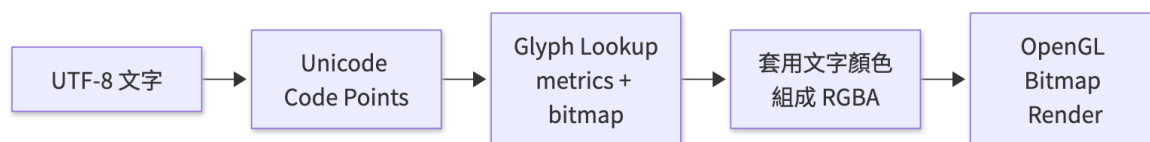


Figure 11: GFNT 執行期文字渲染流程

6 Framebuffer 與輸出

6.1 Color Buffer 與畫面還原

ColorBuffer 以 RGBA 儲存 framebuffer，透過 `glReadPixels()` 擷取畫面，再以 `glDrawPixels()` 還原。Window 分別記錄「內容需要重畫」與「畫面需要重新擷取」兩種狀態：Scene 或版面改變時先重畫 Element tree，畫面穩定後再更新 cache；若只是視窗重新曝光，則直接還原像素。圖 12 顯示狀態轉換，座標、pixel row 與 OpenGL state 的處理詳見附錄 C。

6.2 圖片輸出

輸出圖片時，程式建立與 Canvas 同尺寸的 GraphicsRenderWindow，關閉 Grid，並重用 Canvas Renderer 繪製 Scene。完成後將 framebuffer 由下到上的 RGBA rows 轉為由上到下的 RGB rows，再寫成 PPM P6；因此輸出不包含主視窗的 Grid 與狀態列。PNG 與 JPEG 目前尚未實作。

圖 13a 的實際輸出尺寸為 800×578 。自由曲線與中文字方向正確，且沒有 Grid 與狀態列，可確認圖片輸出只重新繪製 Scene 內容。

7 特殊貢獻摘要

除了基本 drawing panel，我另外完成可判斷儲存狀態的 Undo / Redo、物件選取與可復原刪除、多視窗與 Modal Dialog、自製 GUI framework、Advanced Stroke，以及支援 UTF-8 中文的 GFNT renderer。這些功能分別改善操作完整性、粗線品質與文字能力；核心實作見第 3.5、4.3、4.4 節及附錄 A、B。

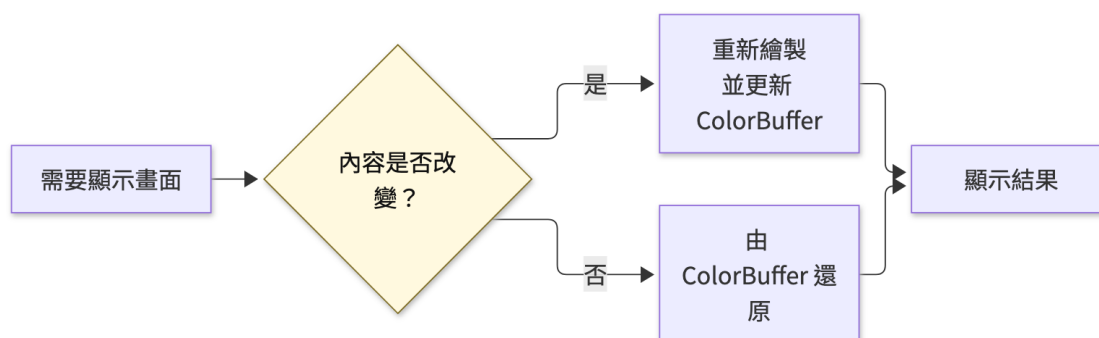


Figure 12: ColorBuffer 快取與畫面還原流程

8 成果展示與作業需求驗證

8.1 繪圖成果

圖 13b 集中呈現 Point、Pencil、Line、Rectangle、Ellipse、Polygon 與 Text 的操作結果，並比較不同 Fill、Stroke、Join、Cap 與 Grid 設定。

8.2 進階功能成果

圖 10 顯示文字輸入，圖 13a 對照 Canvas 與 PPM 輸出。圖 13c 則呈現兩個 PaintWindow 和 Confirm Dialog，用來觀察不同視窗的 Document 是否彼此獨立，以及 Dialog 開啟時背景視窗的輸入狀態。

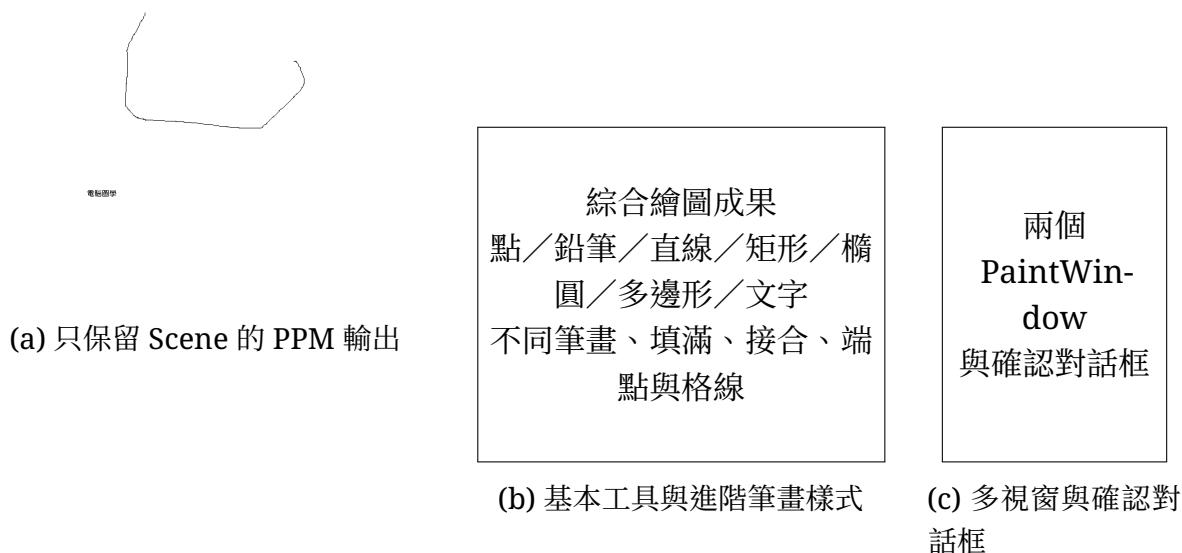


Figure 13: 基本繪圖與進階功能成果

8.3 Requirement Checklist

表 3 依 README 與實際程式碼標示完成狀態。介面中存在但 callback 為空，或 file command 存在但內容未序列化者，都不列為完成。

Table 3: 作業要求與實作狀態

Requirement	Status	Implementation
Interactive 2D drawing panel	已完成	CanvasElement、tool draft、Scene 與 Renderer。

Requirement	Status	Implementation
Hierarchical menus	已完成	FreeGLUT Menu 封裝與 File/Edit/Tools/Stroke/Fill/Point/Text/Grid 子選單。
Keyboard events	已完成	KeyDown/KeyUp/TextInput、InputState 與 ShortcutManager。
Mouse / motion events	已完成	MouseDown/Up/Move、Click/DoubleClick、hit test 與 mouse capture。
Window events	已完成	Close、Reshape、Visibility、Entry callback 轉為 Window Event。
Color	部分完成	Stroke/Fill/Text 預設色可選；Custom color callback 尚未完成。
Line width / point size	已完成	選單與中括號快捷鍵可調整。
Fill mode	已完成	OUTLINE、FILLED、ADVANCED；Advanced 分開處理 Fill 與 Stroke。
Mouse / motion drawing	已完成	DragTool、PencilTool 與 PolygonTool。
Color buffer save / restore	已完成	ColorBuffer 使用 glReadPixels / glDrawPixels 做視窗 cache。
Save color buffer	已完成	PPM P6 export 輸出重新 render 後的 Canvas framebuffer。
Text input	已完成	Modal InputDialog、InputElement、TextObject 與三種字型 renderer。
Grid	已完成	25 px line grid、dot grid 與 none。
Grid-line / scales / positions / sizes	部分完成	已有 line/dot Grid，Status Bar 會顯示游標位置與 Canvas 尺寸；程式中沒有獨立的 scale/ruler。
Undo / Redo	已完成	CommandHistory 與 Add/Insert/Remove/Clear commands。
Multi-window	已完成	Application 管理視窗，Window registry 負責 callback routing；Modal Dialog 開啟時背景視窗停止接收輸入。
Save / Load document	部分完成	.gpt 可寫 header/END 並有 UI/Command；Scene serialization/deserialization 尚未完成。
Selection tool	已完成	可選取最上層圖形或文字、顯示範圍框，並以 Delete/Backspace 刪除；支援 Esc 取消與 Undo/Redo。

Requirement	Status	Implementation
PNG / JPEG export	尚未完成	僅支援 binary PPM。
Fancy ideas	已完成	自製 GUI、advanced stroke、GFNT、UTF-8、多視窗與 framebuffer cache。

9 開發問題與解決方法

9.1 GLUT Callback 與多視窗狀態

FreeGLUT callback 不會附帶 C++ object pointer，同一組 static callback 又可能由不同視窗觸發。程式因此維護 GLUT window ID 到 Window* 的查找表，每次先辨識來源再轉交對應物件；Menu 也以 ID 對應實際動作。這層 routing 讓後續 Event、Element 與 Document 不必依賴全域的「目前視窗」，完整生命週期見附錄 A。

9.2 Draft 與 Undo / Redo

拖曳需要即時預覽，但若每次 Motion 都修改 Scene，Undo history 會累積大量中間狀態。程式將未完成圖形保留在 Tool 的 draft，由 Renderer 畫成 overlay；只有操作完成時，Document 才建立 Command 並修改 Scene。繪圖中按 Undo 會先取消 draft，沒有 draft 時才復原上一筆 Command，使預覽、正式資料與歷史紀錄各自有清楚的生命週期。

9.3 粗線轉角的幾何例外

自行產生粗線 triangle 時，尖角會讓外側交點離頂點過遠，接近 180 度折返或短 segment 也可能造成不穩定交點。程式先移除重複點，再以 cross product 與 dot product 區分情況；MITER 超過 miter limit 時退回 BEVEL，內側交點也設距離上限。求交失敗時則使用不依賴遠端交點的 BEVEL 或 ROUND 畫法，避免快速繪製短折線時出現尖刺與交錯 triangle。

9.4 UTF-8 與 GFNT 資料

FreeGLUT 的一般 keyboard callback 一次只提供一個 byte，若直接用 byte index 編輯，Backspace 可能切斷中文字。輸入框因此以 code point vector 保存文字，顯示時才重新編碼為 UTF-8。GFNT Loader 則在載入時檢查 Header、Glyph Table、Unicode 範圍、排序與 bitmap 邊界，render loop 只需 binary search glyph；格式與驗證流程見附錄 B。

10 已知限制與未來改進

目前最直接的限制是 .gpt 檔案只會寫出檔頭與結尾標記，讀取時也還沒有解析 Scene，因此 Save/Load 不能真正儲存繪圖內容。下一步需要先定義各種 SceneObject 與 Style 的序列化格式，再測試存檔後重新開啟是否能還原相同內容。現階段若要保留畫面，應使用 PPM export。

Select 已支援單一物件的 hit testing、最上層選取、範圍框與可復原的刪除，但還不能移動、縮放或多選；選取框的四個角點目前也只是視覺提示。Custom color 已經出現在 menu 中，但 callback 仍是空的。Renderer 方面，GL_POLYGON 對凹多邊形不可靠，之後可加入 triangulation；Ellipse 固定使用 64 個 segments，也可以依半徑調整。圖片輸出目前只有 PPM，尚未支援 PNG 或 JPEG。Grid、游標位置與 Canvas 尺寸已經能顯示，但程式沒有額外的 scale/ruler。

11 心得

一開始我把這個作業理解成在 GLUT callback 裡切換工具，再直接呼叫 OpenGL 畫圖。這樣做前幾個功能很快，但加入多邊形、文字、Undo/Redo、對話框與多視窗後，callback 會同時負責輸入狀態、Document 修改和 rendering，變得很難判斷一個事件到底改了什麼。因此我逐步拆出 Window、Event、Document、Tool、Command 與 Renderer，讓每一層只處理自己掌握的狀態。

這次最花時間的部分不是基本圖形，而是功能交界處。例如拖曳離開 Canvas 後 MouseUp 還要送回原工具；粗線轉角要處理接近平行與折返；中文字不能用 byte index 移動 caret；多視窗 timer 也不能假設目前 OpenGL context 永遠正確。把這些情況做完後，我比較能理解 GUI framework 為什麼需要 focus、capture、layout 與 event routing，而不是只有幾個 callback。

專案範圍後來確實比 drawing panel 大。如果重做一次，我會先固定最小需求：Scene、幾種基本工具、Command 與一條 Rendering Pipeline，等檔案格式真的能存檔和還原後，再加入自製 GUI 與 GFNT。這也讓我發現，UI 上已經出現功能入口，不代表功能本身就完成了。之後如果再做類似專案，我會更早把資料層、操作流程與 UI 一起完成，而不是先把選單項目做出來。

A 自製 GUI Framework

A.1 設計動機與範圍

這次作業沒有使用第三方 GUI framework，而是依照 drawing panel 的需要自行實作。我的目標不是做一套通用的桌面 toolkit，而是處理這個程式實際用到的多視窗、基本 control、Layout、focus、mouse capture、Event bubbling 與 OpenGL rendering。圖 14 概括 framework 的五個核心角色。

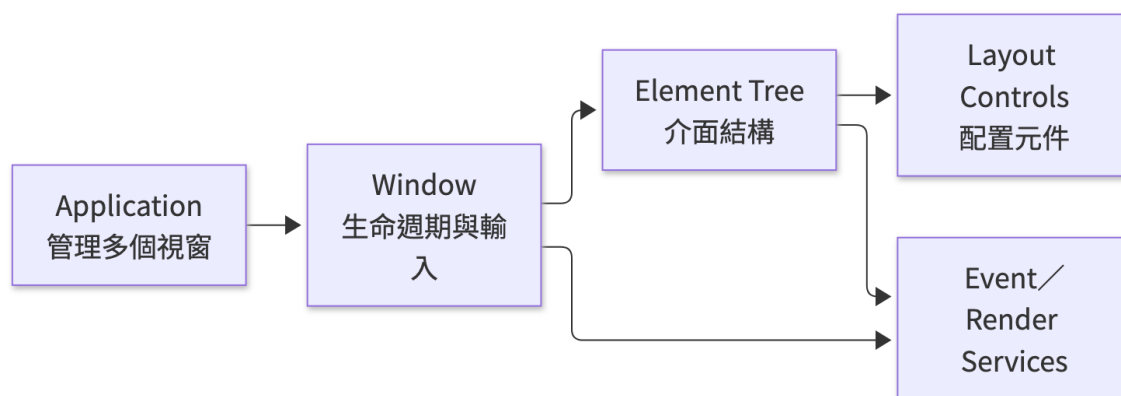


Figure 14: GUI framework 的五個核心角色

A.2 Application 與 Window

Application 建立並管理所有 Window。Window 內部記錄 GLUT window ID、大小、標題、RootElement、ColorBuffer 與互動狀態。每個 Window 都註冊相同的 static callbacks，再透過圖 15 的 ID 查找方式找回正確 instance。PaintWindow、InputDialogWindow、ConfirmWindow 與 GraphicsRenderWindow 都建立在這一層之上。

A.3 Element Tree 與 Layout System

每個 Element 記錄 parent、children、margin、padding、alignment 與 bounds。Layout 分成 measure 和 arrange 兩個階段：measure 先由下而上詢問

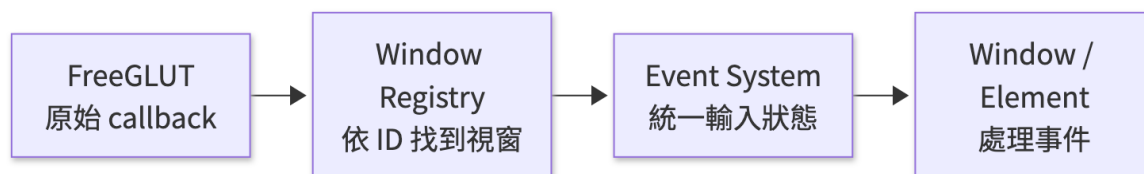


Figure 15: GLUT 回呼的路由流程

每個元件需要多少空間，`arrange` 再由上而下分配實際位置。`RootElement` 只放一個主要內容；`StackPanel` 沿水平或垂直方向排列，並依 `grow` 權重分配剩餘空間；`DockPanel` 依加入順序從四邊扣除空間，最後一個 `child` 取得剩下區域。圖 16 是 `PaintWindow` 實際使用的 UI tree，圖 17 則整理從屬性變更到重新排版與繪製的流程。

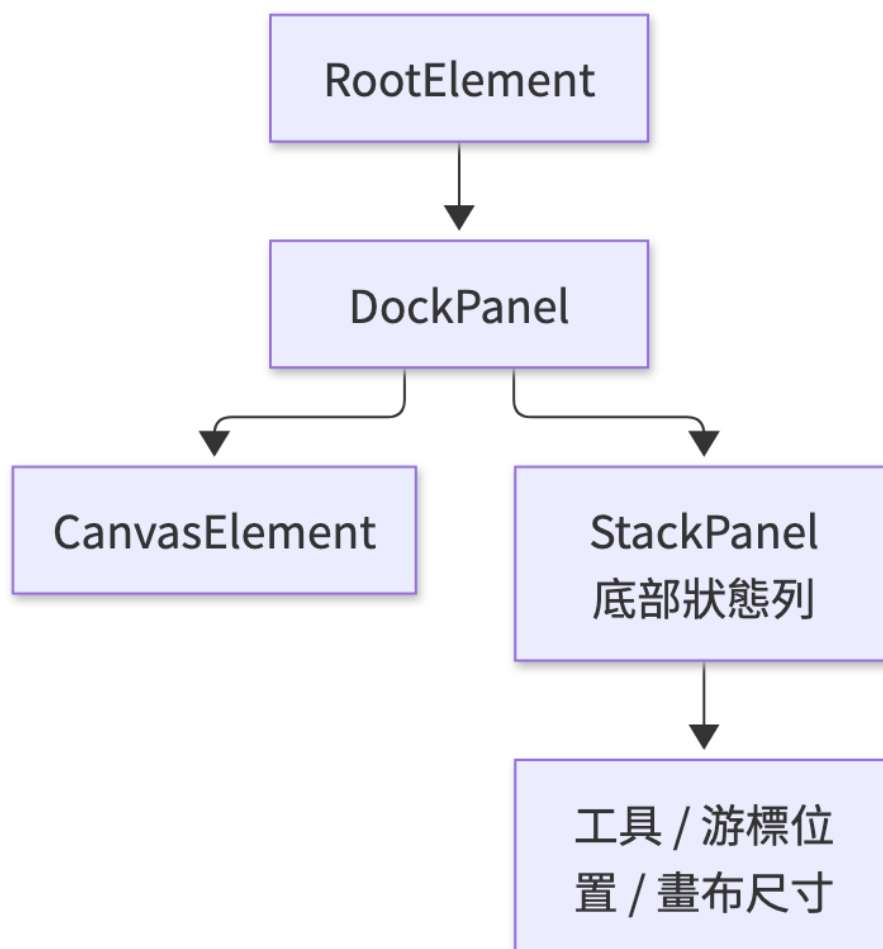


Figure 16: `PaintWindow` 的 Element 與 Layout 樹狀結構



Figure 17: Element Tree 的測量、配置與重新繪製流程

當 `preferred size`、`margin`、`padding` 或 `alignment` 改變時，`Element` 會呼叫 `invalidateLayout()`，再由所屬 `Window` 把 `Layout` 標成需要更新。下一次 `render` 或 `hit test` 前，`Window` 先呼叫 `ensureLayout()`：measure 從 `RootElement` 往子節

點傳入可用空間，子節點算完內容需求後再回報 `desired size`；`arrange` 則從視窗範圍開始，逐層分配實際 `slot` 與 `bounds`。這樣滑鼠命中測試使用的範圍會和畫面上實際繪製的位置一致。

A.4 EventTarget、Bubbling 與 stopPropagation

`EventTarget` 依 `Event` 型別記錄 `listener`。事件開始傳遞時，`target` 會保持為最初接收的元件，`current target` 則隨著 `bubbling` 一層一層改變。`Element` 會把事件傳給 `parent`，`RootElement` 再往上交給 `Window`。輸入框或快捷鍵處理完按鍵後，可以呼叫 `stopPropagation()`，例如避免同一個 `Enter` 同時提交輸入框，又觸發上層 `Window` 的操作。傳遞流程見圖 18。

A.5 Mouse Capture 與 Input State

`Window` 會記錄目前取得 `focus`、`hover` 與 `mouse capture` 的 `Element`。`MouseDown` 後 `Canvas` 取得 `capture`，所以 `Motion` 與 `MouseUp` 仍會送回 `Canvas`；`Hover` 不受 `capture` 影響，繼續依游標實際位置更新。鍵盤狀態由視窗共用，滑鼠狀態則由各 `Window` 記錄。`Modal Dialog` 開啟或滑鼠進出視窗時，程式會清理相關狀態，避免按鍵或滑鼠按鈕一直停在按下狀態。

A.6 Multi-window Lifecycle

關閉 `callback` 不會立刻刪除 `C++ object`，而是先移除 `GLUT ID`，接著送出 `WindowCloseEvent`。`PaintWindow` 可以在這時檢查未儲存的修改並顯示 `Confirm Dialog`；使用者確定關閉後，`Window` 才會標記為可移除，最後由 `Application` 的 `idle callback` 釋放物件。`Timer` 每次執行前也會先確認視窗是否仍存在，避免 `callback` 執行到一半時物件已被銷毀。

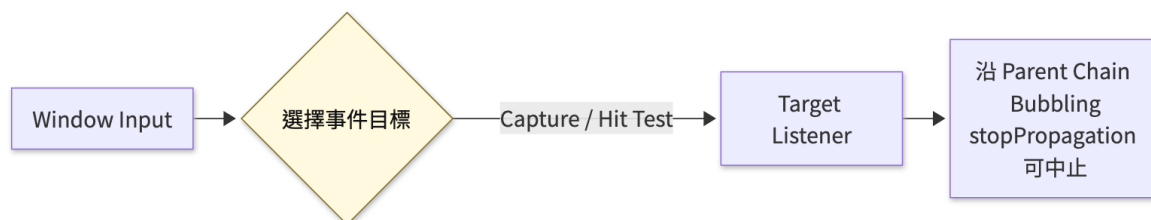


Figure 18: Event 目標選取與沿父節點向上傳遞的流程

B GFNT 自訂點陣字型格式

B.1 設計動機

FreeGLUT 內建 Bitmap/Stroke 字型不足以涵蓋中文，因此程式使用簡單的 GFNT binary resource。格式採用固定大小的 header 與 glyph entry，載入時可以先檢查檔案範圍，之後再依排序過的 Unicode code point 做 binary search。正文的圖 11 已整理文字繪製的主流程；本附錄再分別說明檔案結構、Loader 檢查與找不到字元時的 fallback。

B.2 Binary Layout

格式使用 1-byte packing，並由 C++ 的 static assertion 確認 GfntHeaderV1 為 16 bytes、GfntGlyphV1 為 18 bytes。檔案中的整數欄位一律使用 little-endian。若 Glyph 數量為 N ，Bitmap Data 的檔案 offset 就是 $16 + 18N$ 。圖 19 與表 4 都依照目前程式中的 struct 欄位與大小整理。

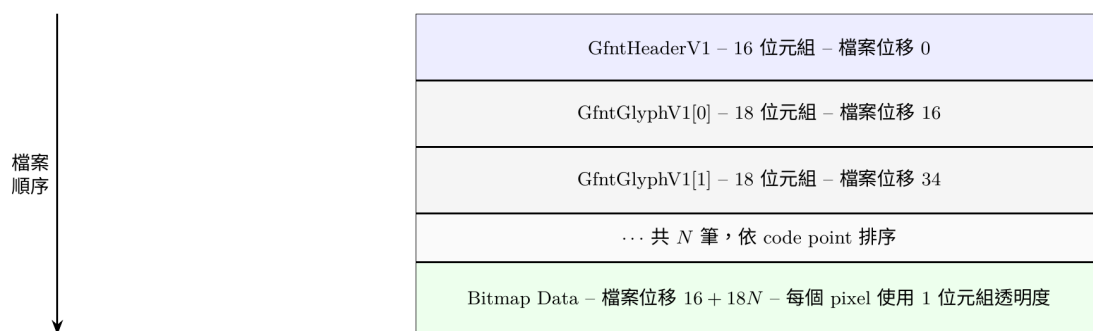


Figure 19: GFNT v1 二進位檔案配置

Table 4: GFNT v1 欄位、相對 offset 與大小

Field	Offset	Size	Meaning
GfntHeaderV1 (16 bytes)			
magic	0	4	ASCII GFNT
version	4	2	格式版本，目前為 1
pixelSize	6	2	nominal pixel size，不可為 0
ascender	8	2	baseline 上方高度，int16_t
descender	10	2	baseline 下方高度，int16_t
glyphCount	12	4	glyph table entry 數量
GfntGlyphV1 (18 bytes each)			

Field	Offset	Size	Meaning
codepoint	0	4	Unicode codepoint
width	4	2	bitmap 寬度
height	6	2	bitmap 高度
bearingX	8	2	pen 到 bitmap 左側的水平位移
bearingY	10	2	baseline 到 bitmap 頂端的垂直位移
advanceX	12	2	下一個 pen X 增量
bitmapOffset	14	4	相對 Bitmap Data 開頭的 byte offset

B.3 Loader 與驗證

Loader 先取得檔案大小，確認至少能容納 16-byte header，再逐欄讀取不同寬度的整數。接著檢查 magic、version、pixel size、ascender、descender，以及 $16+18N$ 是否超出檔案。Bitmap Data 讀完後，程式還會逐一檢查 Glyph：code point 必須按照遞增順序排列，不得超過 U+10FFFF 或落在 surrogate 範圍，而且流程如圖 20 所示。

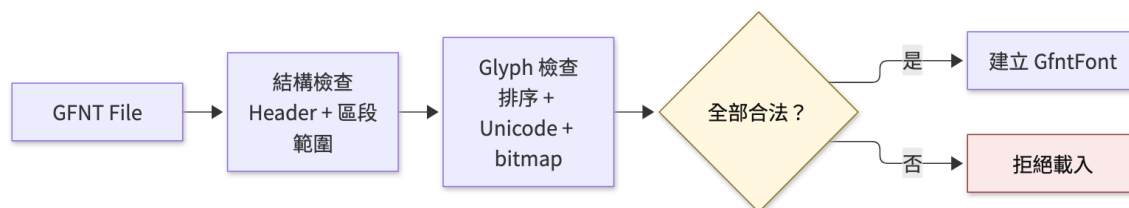


Figure 20: GFNT Loader 的讀取與驗證順序

$$o + w \times h \leq \text{Bitmap Data size.}$$

其中 o 、 w 、 h 分別對應 bitmapOffset、width 與 height。程式實際以減法形式比較，避免加法 overflow。這些檢查都在載入字型時完成，render loop 不需要每畫一個字就重新檢查。

B.4 UTF-8 Decoder 與 Glyph Lookup

UTF-8 decoder 會檢查 sequence 長度、continuation bytes、最小合法值、最大 code point 與 surrogate，遇到錯誤時消耗一個 byte 並回傳 U+FFFD。Glyph Table 在載入時已確認順序，因此查找時使用 `std::lower_bound`；若找不到原字元，再依序嘗試 U+FFFD 與問號。圖 22 對應 `findGlyphOrFallback()` 的實際查找順序；三次都找不到時，Renderer 以 header 的 pixel size 推進 pen，而不繪製 bitmap。

輸入框的 caret index 記錄在 Unicode code point 之間，而不是 UTF-8 byte 之間。圖 21 以「A 你 B」說明兩者差異，因此刪除「你」時會移除一個完整 code point。

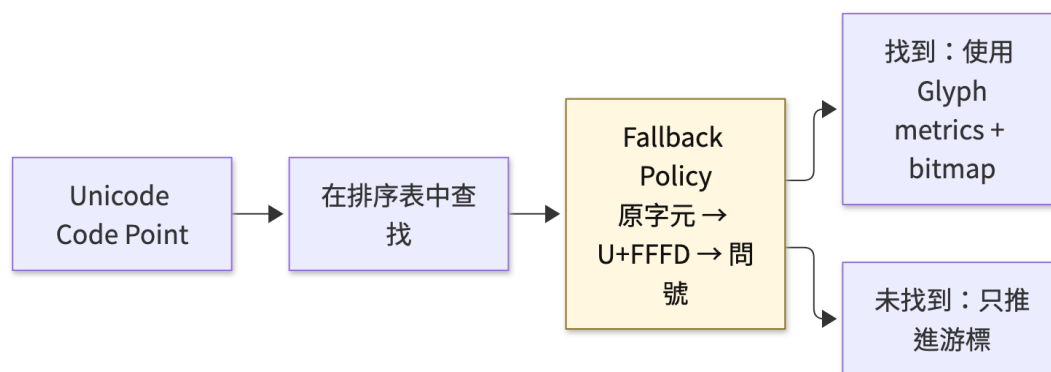


Figure 22: GFNT Glyph 查找與替代字元使用順序

B.5 Bitmap Rendering

TextRenderer 以 header 的 ascender/descender 計算 baseline 與 line height。每個 glyph 的左上角為

$$x = \text{penX} + b_x, \quad y = \text{baselineY} - b_y.$$

其中 b_x 與 b_y 是 Glyph 的 horizontal bearing 與 vertical bearing。Bitmap Data 每個 pixel 儲存 alpha；Renderer 讀取目前的 OpenGL color，組成 RGBA pixels，再開啟一般 alpha blending。因為 Glyph rows 由上到下儲存，繪製時使用負的 Y pixel zoom。完成一個 Glyph 後，penX 再增加 advanceX。整個執行流程已在圖 11 表示。

UTF-8 bytes、code point 與 caret 的對應

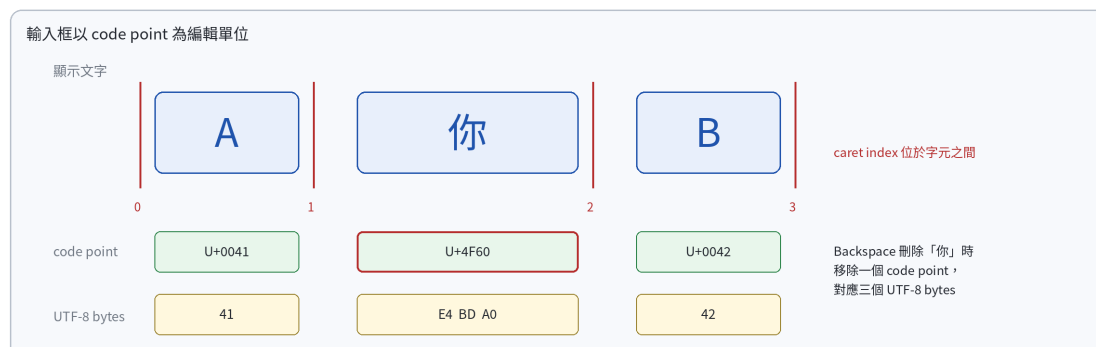


Figure 21: UTF-8 bytes、code point 與 caret index 的對應

圖 23 使用實際的 Cubic-11 與 Unifont 字型繪製「你」(U+4F60)。Cubic-11 雖然名稱包含 11，但本專案轉換時使用的 pixelSize 為 12，因此產生的「你」字 bitmap 為 13×11 pixels，高度是 11 pixels。左半部依 Converter 產生 GFNT 欄位的方式，標出基線、bearing、bitmap 範圍與 advance；右半部則將 Unifont 的 16×16 alpha bitmap 逐格畫出，並顯示 Renderer 如何把目前顏色乘上 Glyph alpha 形成 RGBA pixels。圖中的第一列對應 GFNT Bitmap Data 的起始位置，負的 Y pixel zoom 會讓 `glDrawPixels()` 由 Glyph 上方往下顯示。

GFNT 點陣字形定位與執行期著色



Figure 23: GFNT 字形定位與點陣圖著色；使用 Cubic-11 及 Unifont 16 的實際「你」字點陣圖

C Framebuffer 與輸出實作細節

C.1 座標與 Pixel Row

UI、滑鼠與 Scene 都採左上角原點、Y 軸向下；OpenGL framebuffer 則以左下角為原點，pixel rows 也由下往上排列。擷取 UI 區域時，程式以視窗高度、區域上緣與區域高度換算 `glReadPixels()` 使用的 bottom 座標。這個換算只改變讀取位置，不會翻轉讀回的資料。

C.2 擷取與還原狀態

程式從 single-buffer 的 front buffer 擷取像素。還原前會暫時關閉 blending、depth、stencil、scissor 與 alpha test，並在操作完成後恢復 PACK / UNPACK alignment 與其他 OpenGL state，避免 ColorBuffer 影響後續 Renderer。Timer 擷取多視窗內容時也會先切換到目標 GLUT window，完成後只在原 window ID 仍有效時切回。

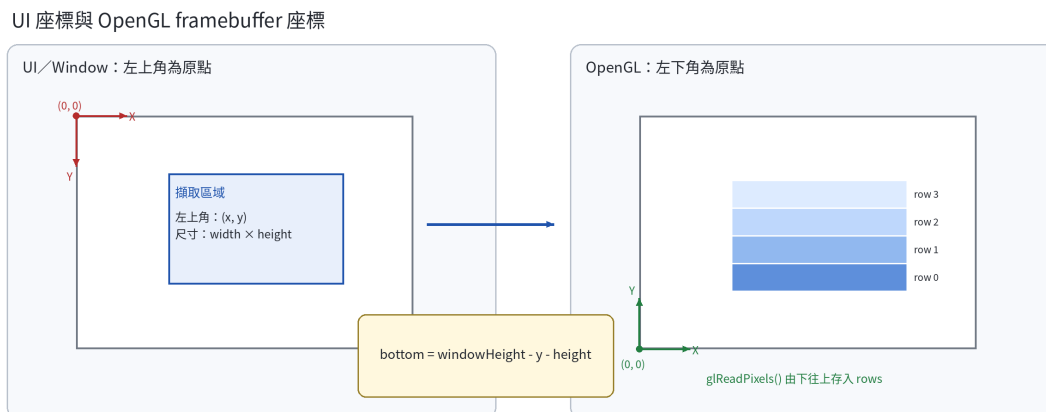


Figure 24: UI 與 OpenGL framebuffer 的座標及 pixel row 順序

C.3 PPM P6 轉換

PpmExporter 由 $\text{height} - 1$ 反向走訪到第 0 列，將 framebuffer 的 bottom-up RGBA 資料轉成 PPM 所需的 top-down RGB bytes，alpha channel 不輸出。圖 25 顯示 row order、header 與 channel 配置。

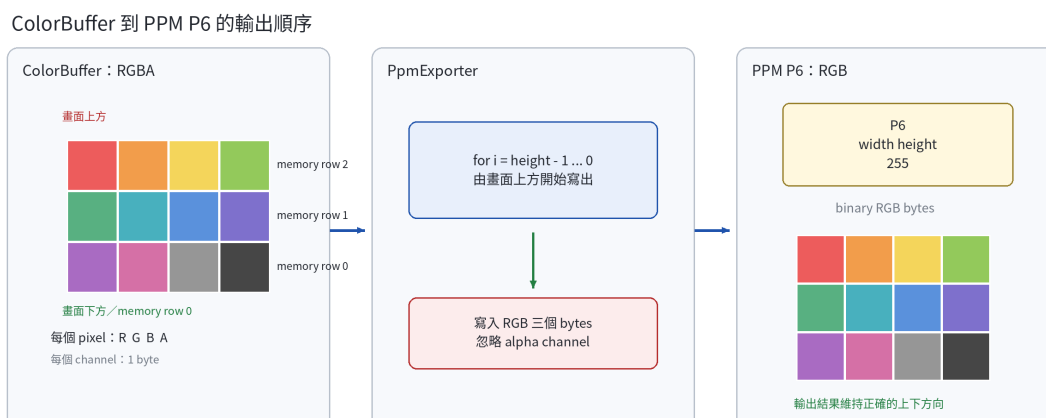


Figure 25: ColorBuffer 轉成 PPM P6 時的列順序與 channel 配置